Heti Menü

Étkezéstervező alkalmazás

Funkcionalitási Specifikáció — v3

Kurzus: **Vibe-coding alapú Webfejlesztési keretrendszerek**
Technológia: **Angular 17+ | Firebase / Firestore**
Dátum: 2025. február

Tartalomjegyzék

1. Áttekintés

1.1 Az alkalmazás célja

A felhasználó saját recepteket hoz létre és böngész, heti menüt állít össze, és automatikusan generált bevásárlólistát kap — mindezt responsive, akadálymentes, jól architekturált Angular webalkalmazásként, Firebase backend-del.

1.2 Technológiai stack

Réteg	Technológia
Frontend	Angular 17+ (standalone components, signals)
Nyelv	TypeScript (strict mode)
Styling	SCSS + CSS custom properties (design tokenek)
Routing	Angular Router (lazy-loaded routes)
Állapotkezelés	Angular Signals + Services
Backend	Google Cloud Firebase
Adatbázis	Cloud Firestore (NoSQL)
Autentikáció	Firebase Authentication (email/jelszó)
Tárhely	Firebase Storage (képek)
Hosting	Firebase Hosting
Tesztelés	Jasmine + Karma (unit), Cypress (E2E)
Build	Angular CLI
Verziókezelés	Git + GitHub

1.3 A 4 fő modul

- Recept Katalógus — receptek böngészése, létrehozása, szerkesztése, törlése
- Heti Menü Tervező — heti naptár nézet, recept hozzárendelés a napokhoz
- Bevásárlólista Generátor — automatikus lista a heti menü alapján
- Felhasználói Profil & Beállítások — auth, profil, preferenciák

2. Kurzus-térkép

Az alábbi táblázat mutatja, hogy az egyes kurzus-hetek hogyan kapcsolódnak a projekt moduljaihoz.

Hét	Kurzus téma	Amit építünk
1.	UX, UI Design & Responsive Layout	Design token rendszer + Recept kártya komponens
2.	Accessibility & Information Architecture	Szemantikus struktúra, ARIA, IA térkép
3.	Platform, Komponensek & Lifecycle	Angular komponensfa, @Input/@Output, lifecycle hooks
4.	Architektura & Navigáció	Angular Router, lazy loading, layout rendszer, guards
5.	Állapotkezelés & Reaktivitás	Signals, Services, RxJS, reactive forms, szűrők
6.	(szünet / puffer)	Félévközi ellenőrzés, konzultáció
7.	Async, Networking & Perzisztencia	Firestore SDK, observables, loading/error, localStorage
8.	Backend integráció & Services	Firestore CRUD, Firebase Storage, service réteg
9.	Security, Authentication & Permissions	Firebase Auth, route guards, Firestore security rules
10.	Testing, Launch	Unit tesztek, E2E, Lighthouse, Firebase Hosting deploy

3. Entitások és adatmodell

3.1 A 6 entitás és CRUD műveleteik

#	Entitás	Create	Read	Update	Delete
1	User	Regisztráció	Profil megtekintés	Profil szerkesztés	Fiók törlése
2	Recipe	Új recept form	Böngészés + részletes nézet	Saját recept szerk.	Saját recept törlése
3	Ingredient	Létrehozás recept formban	Keresés/lista a formban	Név/kategória módosítás	Törlés (ha nincs ref.)
4	Category	Új kategória	Szűrőkben, dropdown-okban	Név módosítás	Törlés (ha nincs ref.)
5	WeeklyMenu	Hét összeállítása	Heti nézet	Slot módosítás	Hét törlése
6	ShoppingList	Generálás heti menüből	Lista nézet	Check, mennyiség, kézi tétel	Lista törlése

3.2 Relációk

Kapcsolat	Típus	Leírás
User → Recipe	1:N	Egy user több saját receptet hoz létre
User → WeeklyMenu	1:N	Egy usernek több hete van
Recipe → Category	N:1	Egy recept egy kategóriába tartozik
Recipe ↔ Ingredient	N:M	Kapcsoló: RecipeIngredient (amount, unit)
Ingredient → Category	N:1	Egy hozzávaló egy bolt-szekcióba tartozik
WeeklyMenu → Recipe	N:M	Slotokon keresztül: nap + étkezés → recipeId
WeeklyMenu → ShoppingList	1:1	Egy héthez egy bevásárlólista
ShoppingList → ShoppingItem	1:N	Egy listában több tétel
ShoppingItem → Ingredient	N:1	Egy tétel egy hozzávalóra hivatkozik

3.3 Firestore kollekciók

A Firestore NoSQL dokumentum-adatbázis, ezért a relációs modellt dokumentum-struktúrára képezzük le. A denormalizált mezők (pl. recipeTitle a slotokban) csökkentik a lekérdezések számát.

users kollekció

- **uid:** string — Firebase Auth uid
- **name, email:** string
- **avatarUrl:** string | null
- **preferences:** { diet[], dailyCalorieGoal, householdSize, measurementSystem, theme }
- **createdAt, updatedAt:** Timestamp

categories kollekció

- **id:** string
- **name:** string — pl. „Vacsora” vagy „Zöldség & Gyümölcs”
- **type:** 'recipe' | 'ingredient' — recept-kategória vs. bolt-szekció
- **icon:** string — emoji
- **sortOrder:** number
- **createdBy:** string | null — userId, vagy null ha rendszer-szintű

ingredients kollekció

- **id:** string
- **name:** string — pl. „Paradicsom”
- **categoryId:** string → categories (type: 'ingredient')
- **defaultUnit:** string — pl. „db”
- **createdBy:** string — userId

recipes kollekció

- **id, title, description, imageUrl:** string
- **categoryId:** string → categories (type: 'recipe')
- **difficulty:** 'könnyű' | 'közepes' | 'haladó'
- **prepTime, calories, servings:** number
- **diet:** string[] — pl. ['vegetáriánus']
- **nutrition:** { protein, carbs, fat }
- **ingredients:** RecipeIngredient[] — beágyazott tömb: { ingredientId, ingredientName, amount, unit }
- **steps:** string[] — elkészítés lépései
- **ownerId:** string — userId

- **isPublic:** boolean

weeklyMenus kollekció

- **id, userId:** string
- **weekStart:** string — ISO dátum (hétfő)
- **slots:** MealSlot[] — { day, mealType, recipeId, recipeTitle, recipeImageUrl, recipePrepTime, recipeCalories }

Megjegyzés: A recept adatok denormalizáltak, hogy ne kelljen 21 külön lekérdezés.

shoppingLists kollekció

- **id, userId, weeklyMenuId:** string
- **items:** ShoppingItem[] — { id, ingredientId, ingredientName, totalAmount, unit, categoryId, categoryName, checked, isManual, fromRecipeIds[] }
- **generatedAt, updatedAt:** Timestamp

4.1 Recept Katalógus

Az app központi modulja. A felhasználó saját recepteket hoz létre, böngészi és szerkeszti őket, valamint részletesen megtekintheti bármelyik receptet.

Recept kártya grid

- Kártyás megjelenítés: kép, kategória badge, cím, leírás, elkészítési idő, kalória, nehézség
- Responsive grid: 1 oszlop (mobile) → 2 oszlop (tablet) → 3 oszlop (desktop)
- Hover effekt desktopon, lazy loading képekre, üres állapot kezelés
- „Saját” badge a felhasználó által létrehozott recepteken

Keresés és szűrés

- Valós idejű debounced keresés (300ms) a recept nevében és leírásában
- Kategória szűrő (dinamikusan a categories kollekcióból)
- Nehézség, elkészítési idő, étrend szűrők (kombinálható, AND logika)
- Forrás szűrő: Összes / Saját / Közösségi
- Aktív szűrők chip-ekben, egyenként törölhető, mobilon bottom sheet panel

Recept részletes nézet (Read)

- Külön oldalra navigál: /recept/:id
- Hero kép, cím, kategória, meta adatok, tápérték információ
- Adag-skálázó: +/– gombok → hozzávalók mennyisége arányosan változik
- Elkészítés lépései, „Hozzáadás a heti menühöz” gomb
- Saját receptnél: Szerkesztés és Törlés gombok

Recept létrehozása (Create)

- Többlépéses form (stepper UI): Alapadatok → Hozzávalók → Elkészítés → Kép → Előnézet
- Hozzávaló kereső/választó (autocomplete az ingredients kollekcióból)
- Új hozzávaló inline létrehozása, ha nem létezik
- Kép feltöltés: drag & drop, preview, max 2MB, jpg/png/webp
- Angular Reactive Forms validáció: kötelező mezők, min. 1 hozzávaló, min. 1 lépés

Recept szerkesztése (Update) és törlése (Delete)

- Route: /recept/:id/szerkesztes — ugyanaz a form, előre kitöltve
- Csak a saját recept szerkeszthető (ownerId === currentUser.uid)
- Törlés: megerősítő dialog, kép törlése Storage-ból, heti menü figyelmeztetés

Érintett kurzus-hetek

Hét	Téma	Kapcsolódás
1.	UX/UI & Responsive	Kártyák, grid, design tokenek, mobile-first
2.	Accessibility & IA	Szemantikus HTML, ARIA, heading hierarchia
3.	Komponensek & Lifecycle	RecipeCard, FilterBar, RecipeForm, Stepper
4.	Architekturá & Navigáció	/recept/:id, /recept/uj, /recept/:id/szerkesztes
5.	Állapotkezelés	Szűrők, keresés, form state, adag-skálázó
7.	Async & Networking	Firestore lekérdezések, loading/error
8.	Backend integráció	Recept CRUD Firestore-ban, kép upload Storage-ba

4.2 Heti Menü Tervező

Egy heti naptár nézet, ahol a felhasználó összeállítja, melyik nap mit eszik. Recepteket rendelhet a napokhoz, áthelyezheti és törölheti őket.

Heti nézet (naptár grid)

- Desktop: 7 oszlop (Hé–Va), minden oszlopban 3 sor: Reggeli / Ebéd / Vacsora
- Tablet: 7 oszlop kompaktabb kártyákkal
- Mobile: vertikális lista, napok egymás alatt, összecsukható accordion
- Aktuális nap vizuális kiemelése

Recept hozzárendelés (Create)

- Üres slot: szaggatott szegélyű placeholder „+ Recept” szöveggel
- Kattintásra: recept-választó modal/drawer (mini lista kereséssel)
- Kitöltött slot: mini kártya (kép, cím, idő, kalória)

Slot módosítás (Update) & törlés (Delete)

- Csere gomb, Törlés gomb (×), Undo lehetőség toast-ból
- Drag & drop: CDK DragDrop desktopon, Áthelyezés opció mobilon

Napi/heti összesités és hét-kezelés

- Napi összesített kalória és elkészítési idő, színkódolás célhoz képest
- Hét-váltó: előre/hátra navigáció, hét másolása, hét törlése
- Automatikus mentés (Firestore real-time), állapot indikátor

Érintett kurzus-hetek

Hét	Téma	Kapcsolódás
1.	UX/UI & Responsive	Naptár grid layout, responsive nézetek
2.	Accessibility	Grid navigáció, ARIA roles a slotokon
3.	Komponensek	WeekView, DayColumn, MealSlot, RecipePicker
4.	Architekturá	/heti-menu route, layout-ba ágyazás
5.	Állapotkezelés	Heti menü state, undo, kalória számítás
7.	Async & Perzisztencia	Firestore real-time, autosave, optimistic update
8.	Backend integráció	WeeklyMenu CRUD Firestore-ban

4.3 Bevásárlólista Generátor

A heti menü alapján automatikusan generált bevásárlólista, amely összevonja az azonos hozzávalókat és csoportosítja bolt-szekciók szerint.

Lista generálása (Create)

- Heti menü recipeId-k → receptek hozzávalóinak lekérdezése
- Azonos ingredientId összevonása, mennyiségek összegzése
- Mértékegység-konverzió ahol lehetséges (3 × 100ml → 300ml)
- Csoportosítás bolt-szekció kategória szerint
- Mentés a shoppingLists kollekcióba

Lista megjelenítés (Read) és interakció (Update)

- Csoportosítva bolt-szekciók szerint (összecsukható accordion)
- Checkbox: kipipálás → áthúzott stílus, szekció aljára kerül
- Kézi tétel hozzáadása: szöveges input + kategória + mennyiség
- Mennyiség módosítás: inline szerkesztés
- Progress bar: „12 / 18 tétel kész” + szekciónkénti mini progress

Törlés (Delete) és export

- Tétel törlése: swipe-to-delete (mobile), delete ikon (desktop)
- Lista nullázása, teljes lista törlése (megerősítő dialog)
- Másolás vágólapra (Clipboard API), Megosztás (Web Share API)
- Nyomtatás: @media print optimalizált layout

Érintett kurzus-hetek

Hét	Téma	Kapcsolódás
2.	Accessibility & IA	Lista struktúra, checkbox ARIA, accordion
3.	Komponensek	ShoppingList, ShoppingSection, ShoppingItem
5.	Állapotkezelés	Checked state, progress számítás
7.	Async & Perzisztencia	localStorage, Clipboard API, Web Share API
8.	Backend integráció	Firestore CRUD, generálás logika

4.4 Felhasználói Profil & Beállítások

Regisztráció, bejelentkezés és profil kezelés. A felhasználó személyre szabhatja az alkalmazást: diétás preferenciák, kalória-cél, háztartás méret, megjelenés.

Regisztráció (User Create)

- Email + jelszó alapú (Firebase Auth createUserWithEmailAndPassword)
- Angular Reactive Forms validáció: email formátum, jelszó erősség, jelszó megerősítés
- Jelszó erősség indikátor (progress bar), inline hibaüzenetek
- Sikeres regisztráció: Firestore users dokument létrejön + redirect főoldalra

Bejelentkezés

- Email + jelszó form (signInWithEmailAndPassword)
- „Emlékezz rám” checkbox (persistence: LOCAL vs. SESSION)
- „Elfelejtett jelszó” flow (sendPasswordResetEmail)
- Hibakezelés: nem áruljuk el, melyik mező hibás (biztonsági ok)

Profil (Read & Update)

- Név, email, jelszó módosítás (reauthenticateWithCredential)
- Profilkép feltöltés (Firebase Storage) + preview + validáció
- Diétás preferenciák, napi kalória cél (slider), háztartás mérete
- Mentés + toast visszajelzés

Beállítások és fiók

- Téma: Világos / Sötét / Rendszer (CSS custom properties váltás)
- Mértékegység: Metrikus / Angolszász
- Kijelentkezés (signOut), Fiók törlése (megerősítő dialog + jelszó)
- Firebase Auth session kezelés: onAuthStateChanged listener

Érintett kurzus-hetek

Hét	Téma	Kapcsolódás
1.	UX/UI & Responsive	Form layout, input design, mobile-optimalizált formok
2.	Accessibility	Form labeling, error messages, aria-describedby
3.	Komponensek	LoginForm, RegisterForm, ProfileEditor, SettingsPanel
4.	Architekturá	/profil, /login, /regisztracio route-ok, AuthLayout
5.	Állapotkezelés	Auth state (signal), form state (reactive forms)
8.	Backend integráció	Firebase Auth, Firestore user CRUD, Storage upload
9.	Security & Auth	Firebase Auth, route guards, Firestore security rules

5. Keresztfunkciós elemek

5.1 Navigáció & Layout

- Fő navigáció: Receptek | Heti menü | Bevásárlólista | Profil
- Mobilon: bottom navigation bar (4 ikon + label)
- Desktopon: felső navbar (logo bal, nav közép, profil avatar jobb)
- Aktív oldal kiemelése (routerLinkActive)
- Layout shell: AppLayoutComponent + AuthLayoutComponent

5.2 Visszajelzések & mikro-interakciók

- Toast notifikációk: sikeres mentés, hiba, figyelmeztetés (auto-eltűnő, 4 mp)
- Loading skeleton: kártyák és listák betöltésekor (nem spinner!)
- Empty state: üres lista/grid → illusztráció + CTA
- Confirmation dialog: destruktív műveleteknél
- Optimistic UI: a felület azonnal frissül, a háttérben megy a Firestore írás

5.3 Accessibility

- Szemantikus HTML: <nav>, <main>, <section>, <article>, <aside>, <header>, <footer>
- ARIA: aria-label, aria-live="polite", aria-describedby (form hibák)
- Keyboard navigáció: Tab, Shift+Tab, Enter, Escape, Arrow keys
- Focus management: modal megnyitás/bezárás, route váltás után fókusz
- Skip link, kontraszt min 4.5:1, prefers-reduced-motion, prefers-color-scheme

5.4 Teljesítmény

- Képek: WebP, loading="lazy", srcset + sizes
- Route-szintű lazy loading (Angular loadComponent / loadChildren)
- Debounce: keresőmezőn (300ms, RxJS debounceTime)
- Firestore: megfelelő indexek, limitált lekérdezések
- Lighthouse cél: 90+ minden kategóriában